TUM
Algorithms for UQ
Start date: June 3rd, 2026
End date: June 17th, 2026

Dr. Vladyslav Fediukov
Iryna Burak
Vikas Kurapati

Exercise 2: Polynomial chaos approximation

Submission

Submit your code and a `.pdf` with your report on Moodle as a `.zip` file named `uq_exercise2_<groupnumber>.zip`. Please add **name** and **matriculation number** of all the group members to the report. Remember to properly describe your approach and analyze the results in the text. Submitting only code is not enough!

1 Lagrange interpolation

In the first assignment we want to look at interpolation and how it can be used in UQ. As you saw in worksheet 5, given a function $f : [-1, 1] \to \mathbb{R}$ and a grid of $N + 1$ interpolation nodes $G = (x_i)_{i=0}^N$, the Lagrange interpolant on this grid reads

$$I_N^G f(x) = L^G(x) \sum_{i=0}^{N} f(x_i) \frac{w_i}{x - x_i}, \tag{1}$$

where

$$L^G(x) = \prod_{i=0}^{N} (x - x_i), \quad w_i = \frac{1}{\prod_{k \neq i}(x_i - x_k)}.$$

formulated in the barycentric form.
To evaluate, e.g., the expected value we can first build a Lagrange interpolant on a coarse grid and later on use Monte Carlo quadrature to estimate the integral:

$$\mathbb{E}[f(X)] = \int_\Omega f(x) p_X(x) dx \approx \int_\Omega I_N^G f(x) p_X(x) dx \approx \frac{1}{M} \sum_{i=1}^{M} I_N^G f(x_i). \tag{2}$$

This has the advantage that we only have N function evaluations on the interpolation grid and later on evaluate the Lagrange interpolant with Monte Carlo.

Assignment 1

Let's revisit our model problem, the linear damped oscillator:

$$\begin{cases} \frac{d^2y}{dt^2}(t) + c\frac{dy}{dt}(t) + ky(t) = f\cos(\omega t) \\ y(0) = y_0 \\ \frac{dy}{dt}(0) = y_1. \end{cases} \tag{3}$$

Considering $t \in [0, 10]$, $c = 0.5$, $k = 2.0$, $f = 0.5$, $y_0 = 0.5$, $y_1 = 0.0$. Assume that $\omega \sim \mathcal{U}(0.95, 1.05)$ and we are interested in $y(10)$ as in previous worksheets. It is enough if you only look at $y_0(10)$, i.e., the first component of $y(10)$.
In this task, we want to write a `python` program to propagate the uncertainty in ω through the model in Eq. (3) using a Lagrange polynomial as surrogate function and compare to the classic Monte Carlo approach.
In detail, build Lagrange interpolants following Eq. (1) using $N = [2, 5, 10, 20]$ interpolation points over the stochastic space of $\omega \sim \mathcal{U}(0.95, 1.05)$. Next, compute the expected value and variance at $y_0(10)$ using each of those four interpolants following Eq. (2) using $M = [10, 100, 1000, 10000]$ Monte Carlo samples. Compare your results to directly using Monte Carlo (without an interpolant) with M samples as in the first programming worksheet. Look again at the relative error using

$$\mathbb{E}_{\text{ref}}[y_0(10)] = [-0.43893703]^T$$
$$\mathbb{V}_{\text{ref}}[y_0(10)] = [0.00019678]^T.$$

as reference solution.
Also implement some time measurements using, e.g., the python `time` package, to compare the runtime of the different approaches.
Hint: Use Chebyshev instead of uniform interpolation points. For an arbitrary interval $[a, b]$*, they can be computed as* $0.5(a+b)+0.5(b-a)\cdot\cos((2i-1)/(2.*N)\cdot\pi), i = 1, ..., N$.
Hint: Fix the seed such that you use the same Monte Carlo samples for computing the quadrature using the Lagrange interpolation model and standard Monte Carlo.
Report the relative error (as table or plot) compared to the reference solution and the timings of the different approaches. What do you observe for the error for increasing number of interpolation points N? State briefly how this approach can be advantageous compared to classic Monte Carlo. Upload the code as `assignment_1.py`.

2 Orthogonal polynomials

In the lecture, we saw that the generalized polynomial chaos (gPC) approximation is defined in terms of orthogonal polynomials. Given a weight function ρ, two polynomials $\phi_i(x)$ and $\phi_j(x)$ are called orthogonal with respect to ρ if

$$< \phi_i(x), \phi_j(x) >_\rho := \int \phi_i(x)\, \phi_j(x)\, \rho(x)\, dx = \gamma_i\, \delta_{ij},$$

where $\gamma_i =< \phi_i(x), \phi_i(x) >_\rho \in \mathbb{R}$ and δ_{ij} is the Kronecker delta function.
Two polynomials $\phi_i(x)$ and $\phi_j(x)$ are called orthonormal with respect to ρ if

$$< \phi_i(x), \phi_j(x) >_\rho := \int \phi_i(x)\, \phi_j(x)\, \rho(x)\, dx = \delta_{ij}.$$

Note that orthogonal polynomials can be easily transformed into orthonormal polynomials via

$$\phi_i(x) := \frac{\phi_i(x)}{\sqrt{\gamma_i}}.$$

A scheme to compute orthogonal polynomials is Stieltjes' three-terms recursion relation

$$\begin{aligned} \phi_{-1}(x) &\equiv 0 \\ \phi_0(x) &\equiv 1 \\ \phi_{n+1}(x) &= (A_n x + B_n)\, \phi_n(x) \,-\, C_n\, \phi_{n-1}(x) \qquad n \geq 0, \end{aligned}$$

where A_n, B_n, C_n are some constants.
Fortunately, we do not need to implement this scheme from scratch. In `chaospy`, it is easy to generate orthogonal polynomials based on a given probability distribution ρ — check out the function `chaospy.generate_expansion`.

Assignment 2.1

Show that, as stated in the introduction above, we can check if two polynomials are orthogonal or even orthonormal by computing the expected value

$$\mathbb{E}[\phi_i(X) \cdot \phi_j(X)]. \tag{4}$$

Use the definitions from above.
Hint: you can use the chaospy.E function to compute the expectation of the product.
Report the derivation in the pdf.

Assignment 2.2

Let $\rho_1 = \mathcal{U}(-1, 1)$ and $\rho_2 = \mathcal{N}(5, 1)$. Moreover, let $N = 10$. Write a `python` program to generate orthonormal polynomials of degree up to N with respect to ρ_1 and ρ_2. Check numerically if they are orthonormal. To visualize the results, show the difference between the obtained inner products and the identity matrix.
Report your results and upload your code as `assignment_2.2.py`

3 Probabilistic collocation: the pseudo-spectral approach

In the methodology that follows, the assumption is that the weight function ρ is a probability density function. In the lecture, we saw that, given a function $f(t, \omega)$, where t denotes the deterministic inputs and ω denotes the stochastic inputs, the degree N gPC approximation of f reads

$$f(t, \omega) \approx f^N(t, \omega) = \sum_{i=0}^{N-1} \hat{f}_i(t)\ \phi_i(\omega), \tag{5}$$

where $\phi_i(\omega), \quad i = 0, \ldots, N-1$ are orthogonal polynomials with respect to the probability distribution ρ of ω and $\hat{f}_i(t)$ are the expansion's coefficients. In this tutorial, we compute $\hat{f}_i(t)$ via

$$\hat{f}_i(t) = \int_{\text{supp}(\rho)} f(t, \omega)\, \phi_i(\omega)\, \rho(\omega)\, d\omega \approx \sum_{k=0}^{K-1} f(t, x_k)\, \phi_i(x_k) w_k, \tag{6}$$

where $\{x_k, w_k\}_{k=0}^{K-1}$ denotes the set of nodes and weights associated to ρ.
For simplicity, we assume that the orthogonal polynomials are orthonormal. After we obtain the gPC expansion coefficients, statistics such as expectation and variance can be easily computed via

$$\mathbb{E}[f(t, \omega)] = \hat{f}_0(t), \quad \text{Var}[f(t, \omega)] = \sum_{i=1}^{\infty} \hat{f}_i^2(t) \approx \sum_{i=1}^{N-1} \hat{f}_i^2(t). \tag{7}$$

Assignment 3

For fixed t, show that the mean of $f^N(t, \omega) = \sum_{i=0}^{N-1} \hat{f}_i(t)\ \phi_i(\omega)$ is given as

$$\mathbb{E}[f^N(t, \omega)] = \hat{f}_0(t) \tag{8}$$

and its variance as

$$\mathrm{Var}[f^N(t,\omega)] = \sum_{i=1}^{N-1} \hat{f}_i^2(t). \tag{9}$$

Hint: $\phi_0(\omega) = 1$
Report the derivations in the pdf.

4 The pseudo-spectral approach in Chaosoy

It is straightforward to implement the gPC + pseudospectral approach in `chaospy`. Check out the functions `chaospy.generate_expansion`, `chaospy.generate_quadrature`, and `chaospy.fit_quadrature` to implement the pseudo-spectral approach.

Assignment 4

Consider the model problem, the linear damped oscillator

$$\begin{cases} \frac{d^2y}{dt^2}(t) + c\frac{dy}{dt}(t) + ky(t) = f\cos(\omega t) \\ y(0) = y_0 \\ \frac{dy}{dt}(0) = y_1. \end{cases} \tag{10}$$

Considering $t \in [0, 10]$, $c = 0.5$, $k = 2.0$, $f = 0.5$, $y_0 = 0.5$, $y_1 = 0.0$, use the `odeint`[1] function from `scipy.integrate` to discretize the model from eq. (10). For the upcoming experiments, the output that we are interested in is again $y_0(10)$. Assume that $\omega \sim \mathcal{U}(0.95, 1.05)$. Write a `python` program to propagate the uncertainty in ω through the model in eq. (10) using the gPC expansion method from eq. (5). Compute the expansion coefficients from eq. (6) using the pseudo-spectral approach. Here, use two approaches:

1. Write your own implementation of the pseudo-spectral approach by computing the integral with the Gaussian quadrature rule directly as in eq. (6).

2. Use the functionalities offered by `chaospy` to compute the coefficients (which uses eq. (6) under the hood).

Consider $K = N = [1, 2, 3, 4, 5, 6]$. Based on these coefficients, compute the expectation and the variance of $y_0(10)$.

[1] see `https://docs.scipy.org/doc/scipy-0.17.0/reference/generated/scipy.integrate.odeint.html`

Compare to a Monte Carlo reference solution computed with $N_{\text{ref}} = 1000000$ samples which is given as

$$\mathbb{E}_{\text{ref}}[y_0(10)] = [-0.43893703]^T$$
$$\mathbb{V}_{\text{ref}}[y_0(10)] = [0.00019678]^T.$$

by computing the relative error.
Report a comparison of the two approaches for the expectation and variance over K by computing the relative error. You can plot the result or report it in a table. Upload your code as `assignment_4.py`.